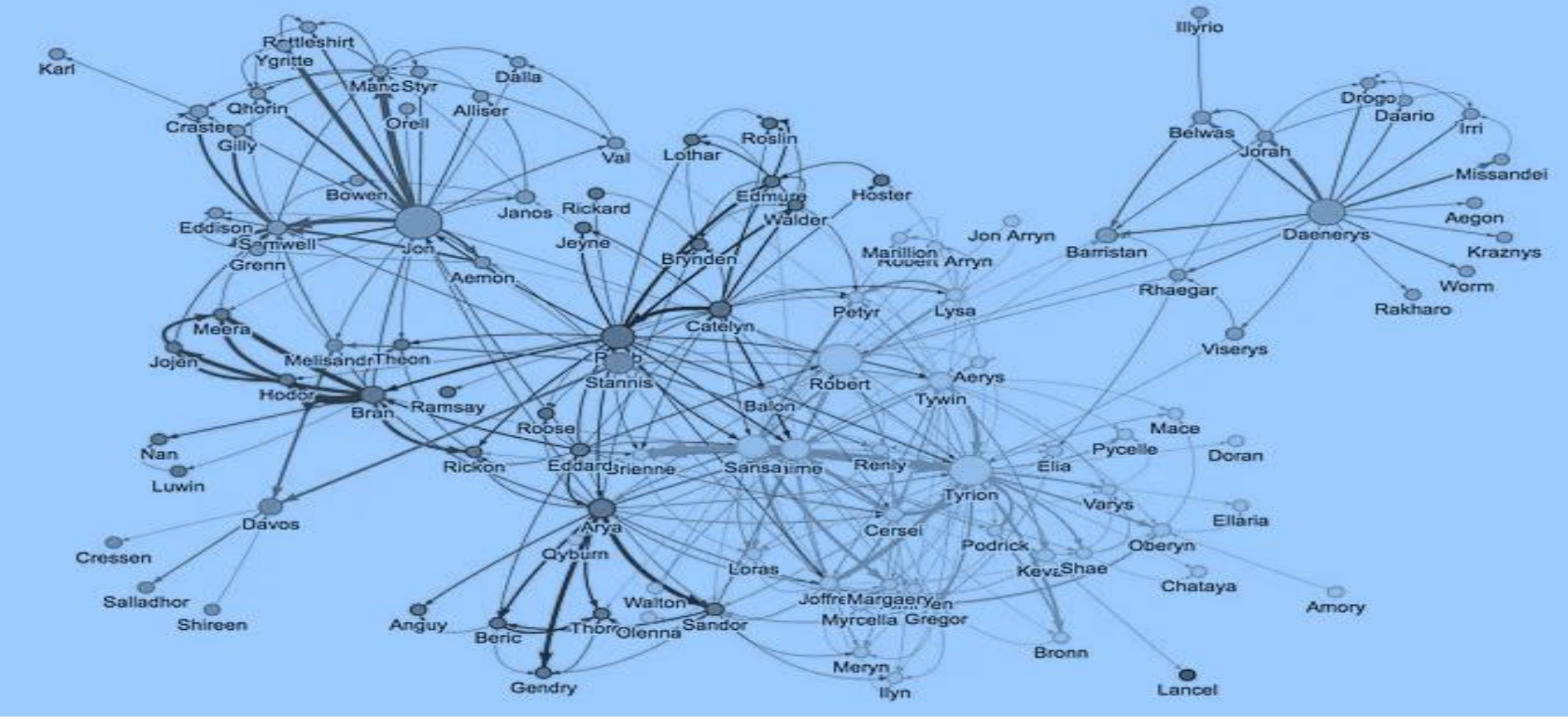


Data Structures

CPT102 – Lecture 11

Erick Purwanto

Xi'an Jiaotong-Liverpool University

西交利物浦大学

CPT102 Data Structures

Lecture 11

Solving Graph Problems

Welcome!

- Welcome to Lecture 11 !
- In this lecture we are going to
  - learn about connected components in undirected graphs
    - solve it using DFS
  - learn about topological sort in DAG
    - solve it using DFS
  - learn about strongly-connected components in directed graphs
    - solve it using Kosaraju algorithm

Review: Depth First Paths using DFS

```
public class DepthFirstPaths {
   private boolean[] marked;
   private int[] edgeTo;
   private int s;

   public DepthFirstPaths(Graph G, int s) {
        ...
        dfs(G, s);
   }
   private void dfs(Graph G, int v) {
      marked[v] = true;
      for (int w : G.adj(v)) {
         if (!marked[w]) {
            edgeTo[w] = v;
            dfs(G, w);
         }
      }
   }
   ...
```

recursive DFS visits unmarked nodes

Review: Breadth First Paths using BFS

```
public class BreadthFirstPaths {
  private boolean[] marked;
  private int[] edgeTo;
  ...

  private void bfs(Graph G, int s) {
     Queue<Integer> queue =
                  new Queue<Integer>();
     queue.enqueue(s);
     marked[s] = true;
     while (!queue.isEmpty()) {
        int v = queue.dequeue();
        for (int w : G.adj(v)) {
           if (!marked[w]) {
              queue.enqueue(w);
              marked[w] = true;
              edgeTo[w] = v;
           }
        }
     }
  } ...
```

BFS uses a queue to visit the nodes level by level

Connectivity Queries

- Vertices v and w are **connected** if there is a path between them
- Goal: Preprocess graph to answer queries of the form **is v connected to w?**
  - in **constant time**

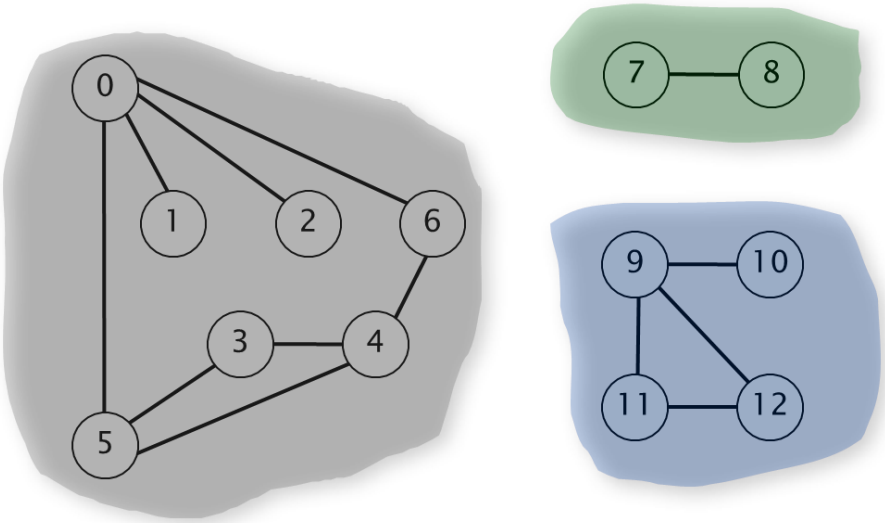
public class CC	
CC(Graph G)	*find connected components in G*
boolean connected(int v, int w)	*are v and w connected?*
int count()	*number of connected components*
int id(int v)	*component identifier for v (between* `0` *and* `count() - 1`*)*

In-Class Quiz 1

- Vertices v and w are **connected** if there is a path between them
- Goal: Preprocess graph to answer queries of the form **is v connected to w?**
  - in **constant time**
- Can we just use the **Union-Find** data structure to achieve the goal above? **Yes / No**
  - why we can / why we cannot?

Part 1: Connected Components

- A **connected component** is a maximal set of connected vertices
  - given connected components, can answer queries in **constant time**



3 connected components

v	id[]
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	1
8	1
9	2
10	2
11	2
12	2

In-Class Quiz 2

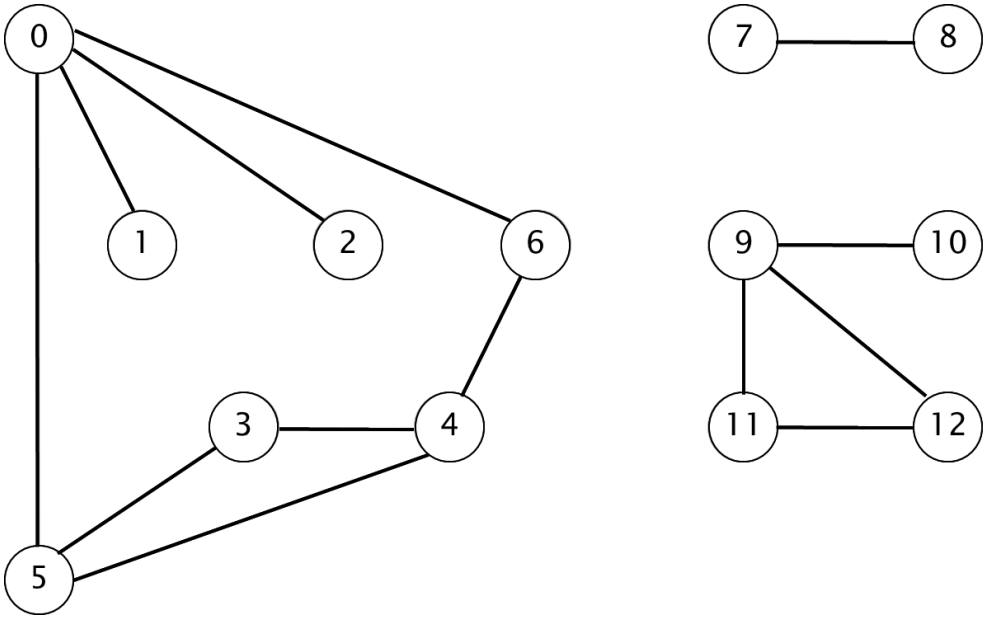
- A **connected component** is a maximal set of connected vertices
  - given connected components, can answer queries in **constant time**
- How do we answer the queries?
  - and, why do they run in constant time?

Connected Components Algorithm

- Connected components algorithm:
  - initialize all vertices v as unmarked
  - for each **unmarked** vertex v,
    run **DFS** to identify all vertices discovered as part of the same component
  - To visit a vertex v:
    - mark vertex v as visited
    - recursively visit all unmarked vertices adjacent to v

Connected Components Demo

- To visit a vertex v:
  - Mark vertex v as visited
  - Recursively visit all unmarked vertices adjacent to v



v	marked[]	id[]
0	F	–
1	F	–
2	F	–
3	F	–
4	F	–
5	F	–
6	F	–
7	F	–
8	F	–
9	F	–
10	F	–
11	F	–
12	F	–

Connected Components Implementation

```
public class CC {
   private boolean[] marked;
   private int[] id;
   private int count;

   public CC(Graph G) {
        marked = new boolean[G.V()];
        id = new int[G.V()];
        for (int v = 0; v < G.V(); v++) {
            if (!marked[v]) {
                dfs(G, v);
                count++;
   } } }

   private void dfs(Graph G, int v) {
      marked[v] = true;
      id[v] = count;
      for (int w : G.adj(v)) {
         if (!marked[w]) {
            dfs(G, w);
   } } } ...
```

run DFS from one vertex in each component

all vertices discovered in same call of DFS have same id

In-Class Quiz 3

```
public class CC {
   private boolean[] marked;
   private int[] id;
   private int count;

   public CC(Graph G) {
        marked = new boolean[G.V()];
        id = new int[G.V()];
        for (int v = 0; v < G.V(); v++) {
            if (!marked[v]) {
                dfs(G, v);
                count++;
   } } }

   private void dfs(Graph G, int v) {
      marked[v] = true;
      id[v] = count;
      for (int w : G.adj(v)) {
         if (!marked[w]) {
            dfs(G, w);
   } } } ...
```

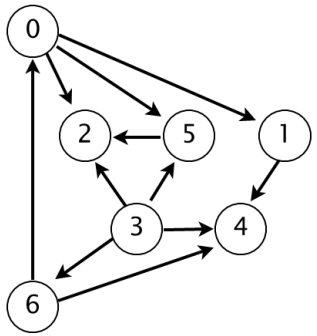
- What is the purpose of the integer variable **count**?

Precedence Scheduling

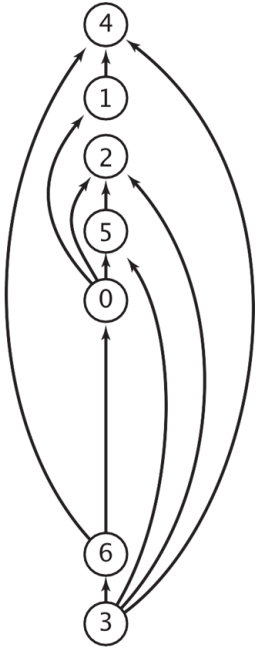
- Given a set of tasks to be completed with *precedence constraints*, in **which order** should we schedule the tasks?
- A **directed** graph: vertex = task, and edge = precedence constraint

1. Algorithms
2. Complexity Theory
3. Artificial Intelligence
4. Intro to CS
5. Cryptography
6. Scientific Computing
7. Advanced Programming

tasks



precedence constraint graph



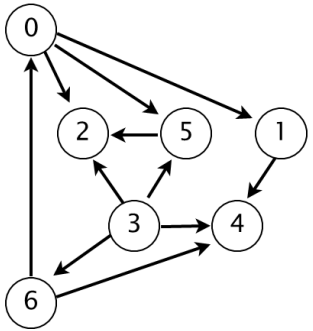
feasible schedule

Part 2: Topological Sort

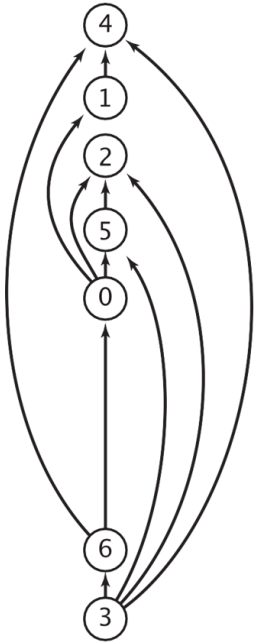
- Topological sort works on Directed Acyclic Graph (DAG)
  - redraw DAG so all edges point upward

0→5	0→2
0→1	3→6
3→5	3→4
5→2	6→4
6→0	3→2
1→4	

directed edges



DAG



topological order

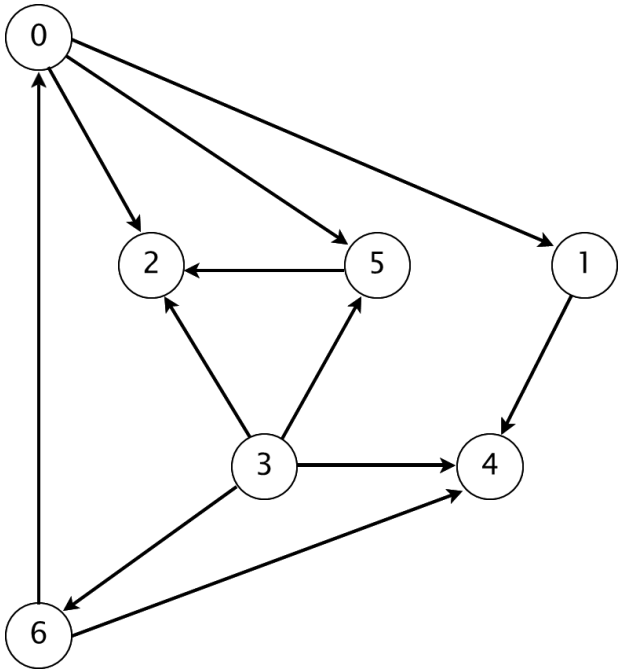
In-Class Quiz 4

- Topological sort works on Directed Acyclic Graph (DAG)
  - redraw DAG so all edges point upward

- Why we **cannot** find vertices in topological order on a **directed cyclic graph**?

Topological Sort Demo

- Run DFS, return vertices in **reverse postorder**

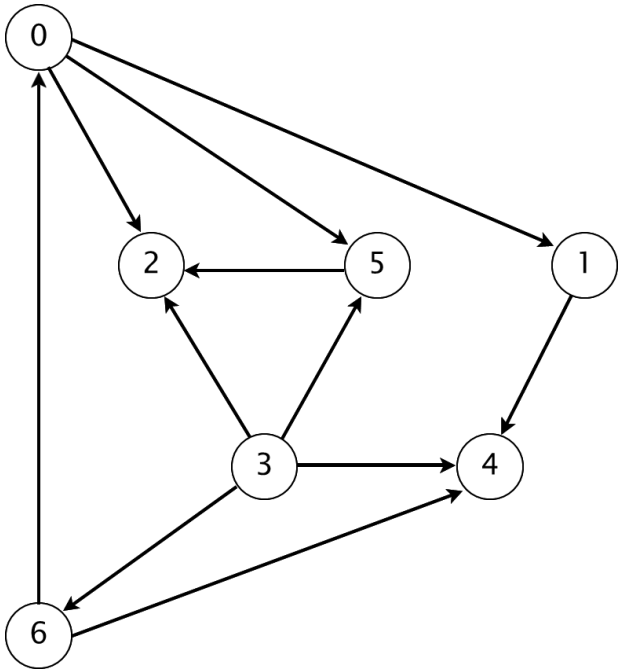


- Visit 0, 1, 4
- Done with 4, 1
- Visit 2
- Done with 2
- Visit 5
- Done with 5
- Done with 0
- Visit 3, 6
- Done with 6, 3

postorder

Topological Sort Demo

- Run DFS, return vertices in **reverse postorder**
  - when we are done with a vertex, put it in a stack, the pop-ed contents are in are in reverse postorder



postorder

4 1 2 5 0 6 3

topological order

3 6 0 5 2 1 4

Topological Sort Implementation

```
public class TopologicalSort {
   private boolean[] marked;
   private Stack<Integer> reversePostorder;

   public TopologicalSort(Graph G) {
        marked = new boolean[G.V()];
        reversePostorder = new Stack<>();
        for (int v = 0; v < G.V(); v++) {
            if (!marked[v])
                dfs(G, v);
   } }

   private void dfs(Graph G, int v) {
      marked[v] = true;
      for (int w : G.adj(v))
         if (!marked[w])
            dfs(G, w);
      reversePostorder.push(v);
   } }
   ...
```

to store the topological sort, create a method to return it

push the vertex v visited by DFS into the stack

In-Class Quiz 5

```
public class TopologicalSort {
   private boolean[] marked;
   private Stack<Integer> reversePostorder;

   public TopologicalSort(Graph G) {
        marked = new boolean[G.V()];
        reversePostorder = new Stack<>();
        for (int v = 0; v < G.V(); v++) {
            if (!marked[v])
                dfs(G, v);
   } }

   private void dfs(Graph G, int v) {
      marked[v] = true;
      for (int w : G.adj(v))
         if (!marked[w])
            dfs(G, w);
      reversePostorder.push(v);
   } }
   ...
```

- The vertex v is added to the stack *outside* the for loop (and *outside* the if), is this correct?
  - why / why not?

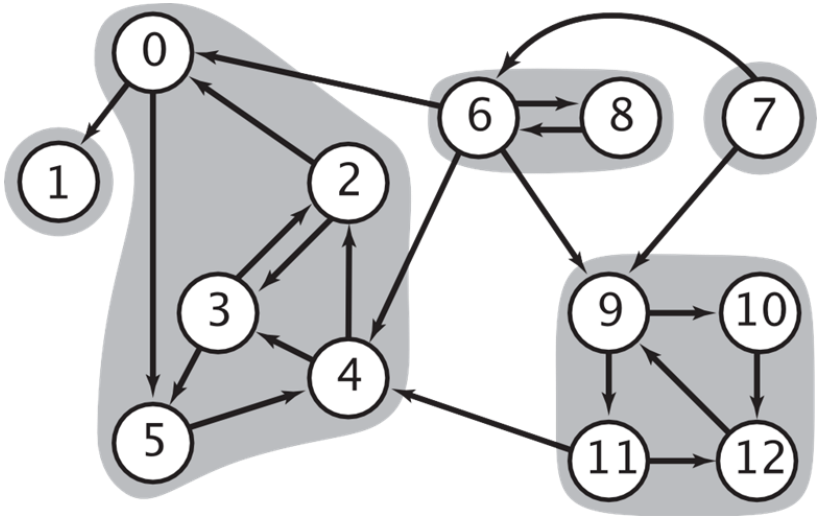
Depth-first Search Orders

- DFS visits each vertex *exactly once*
  - the order in which it does so can be important
    - **preorder**: order in which dfs() is called
    - **postorder**: order in which dfs() returns
    - **reverse postorder**: reverse order in which dfs() returns

```
private void dfs(Graph G, int v) {
   marked[v] = true;
   preorder.enqueue(v);
   for (int w : G.adj(v))
      if (!marked[w])
         dfs(G, w);
   postorder.enqueue(v);
   reversePostorder.push(v);
}
```

Part 3: Strongly-connected Components

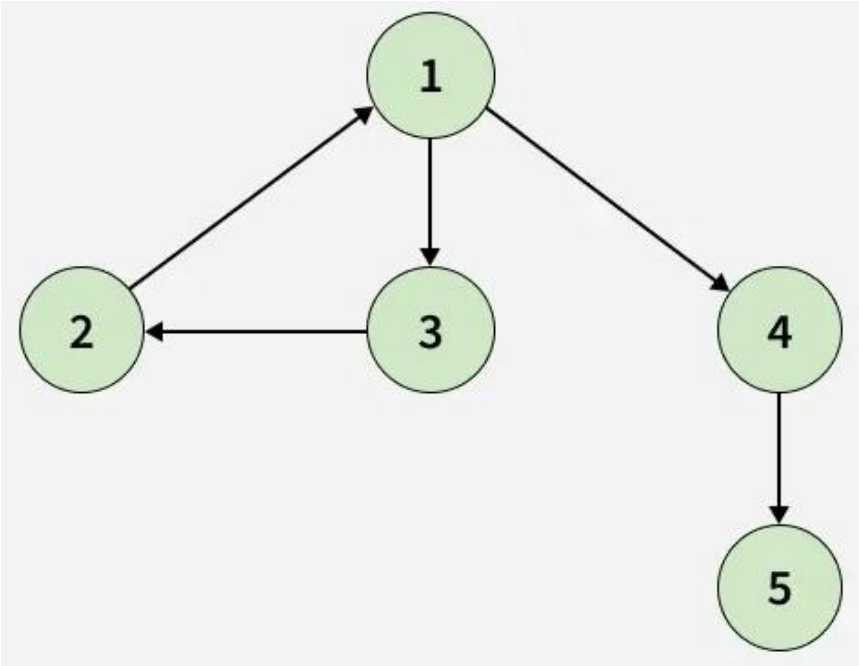
- Vertices v and w are **strongly connected** if there is **both** a ***directed*** path from v to w **and** a ***directed*** path from w to v
- A **strong component** is a maximal subset of strongly-connected vertices



5 strongly-connected components

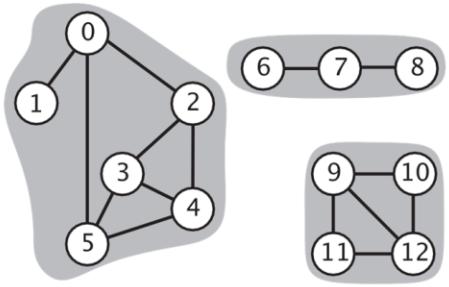
In-Class Quiz 6

- How many **strongly-connected components** in the graph below? Please explain.



Connected components

v and w are connected if there is a path between v and w



3 connected components

connected component id (easy to compute with DFS)

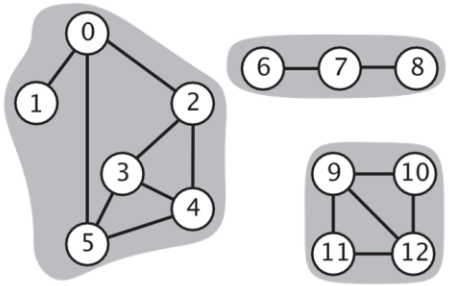
	0	1	2	3	4	5	6	7	8	9	10	11	12
id[]	0	0	0	0	0	0	1	1	1	2	2	2	2

```
public boolean connected(int v, int w)
{  return id[v] == id[w];  }
```

constant-time client connectivity query

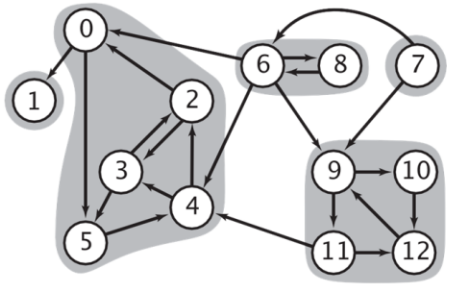
Connected components vs strongly-connected components

v and w are connected if there is a path between v and w



3 connected components

v and w are strongly connected if there is both a directed path from v to w and a directed path from w to v



5 strongly-connected components

connected component id (easy to compute with DFS)

	0	1	2	3	4	5	6	7	8	9	10	11	12
id[]	0	0	0	0	0	0	1	1	1	2	2	2	2

```
public boolean connected(int v, int w)
{  return id[v] == id[w];  }
```

constant-time client connectivity query

strongly-connected component id (how to compute?)

	0	1	2	3	4	5	6	7	8	9	10	11	12
id[]	1	0	1	1	1	1	3	4	3	2	2	2	2

```
public boolean stronglyConnected(int v, int w)
{  return id[v] == id[w];  }
```

constant-time client strong-connectivity query

History of Strongly-connected Components Algorithms

1960s: Core OR problem.

- Widely studied; some practical algorithms.
- Complexity not understood.

1972: linear-time DFS algorithm (Tarjan).

- Classic algorithm.
- Level of difficulty: high
- Demonstrated broad applicability and importance of DFS.

1980s: easy two-pass linear-time algorithm (Kosaraju-Sharir).

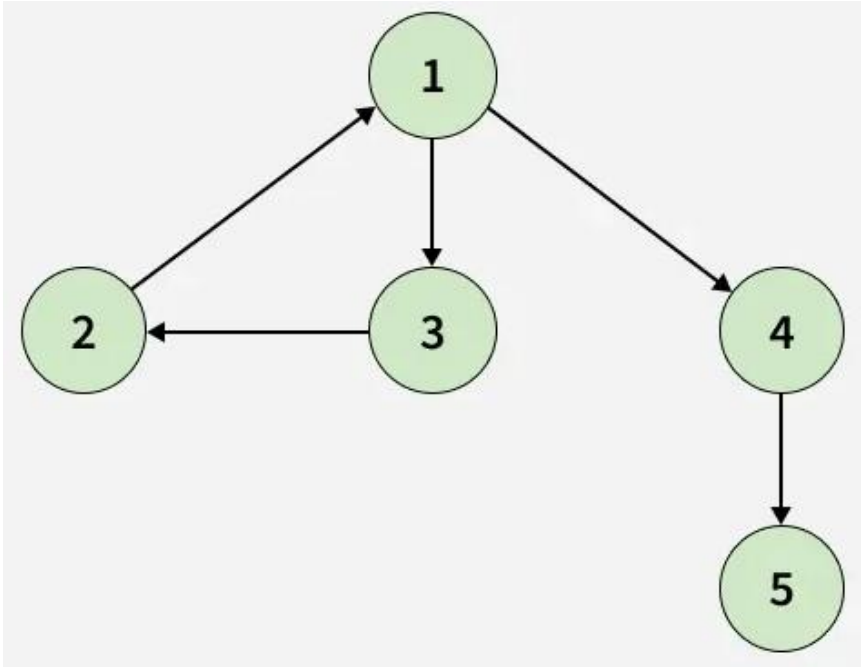
- Forgot notes for lecture; developed algorithm in order to teach it!
- Later found in Russian scientific literature (1972).

1990s: more easy linear-time algorithms.

- Gabow: fixed old OR algorithm.
- Cheriyan-Mehlhorn: needed one-pass algorithm for LEDA.

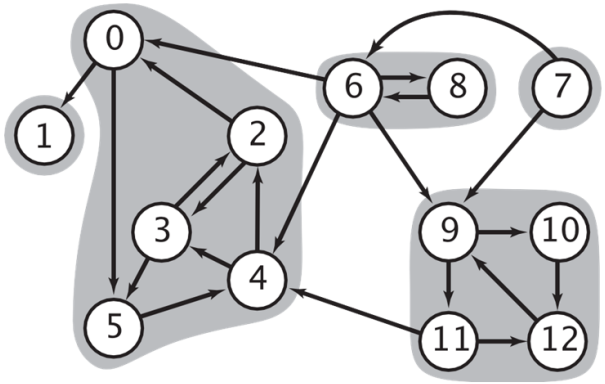
In-Class Quiz 7

- If we reverse the direction of the edges in a graph, will we get the same **strongly-connected components** ? Please explain.

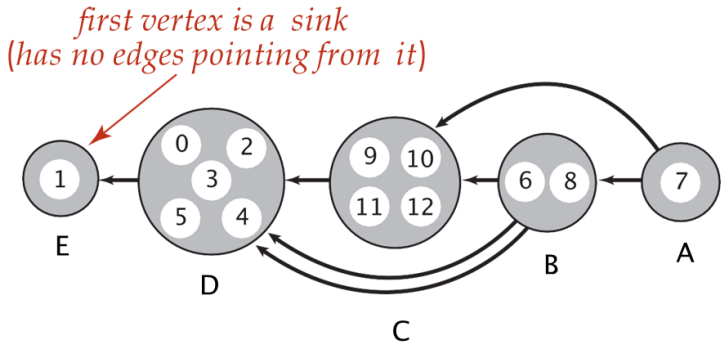


Kosaraju Algorithm's Intuition

- Notice that the strong components in G are same as in G^R (G's reverse graph)
- Kernel DAG: contract each strong component into a single vertex



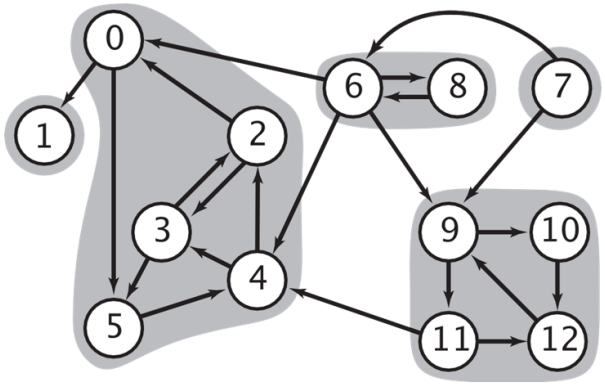
digraph G and its strong components



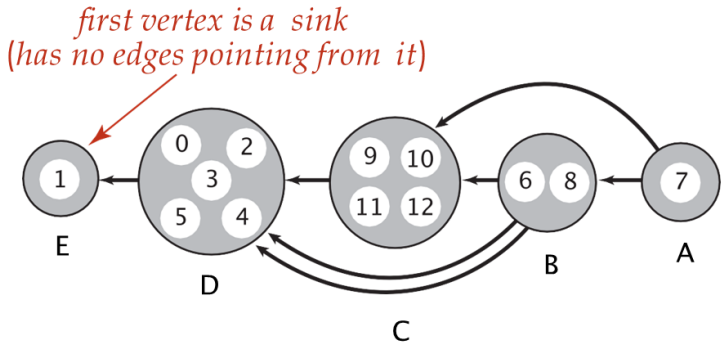
kernel DAG of G (topological order: A B C D E)

Kosaraju Algorithm's Intuition

- Notice that the strong components in G are same as in G^R (G's reverse graph)
- Kernel DAG: contract each strong component into a single vertex
- Idea:
  - compute topological order (reverse postorder) in kernel DAG
  - run DFS, considering vertices in reverse topological order



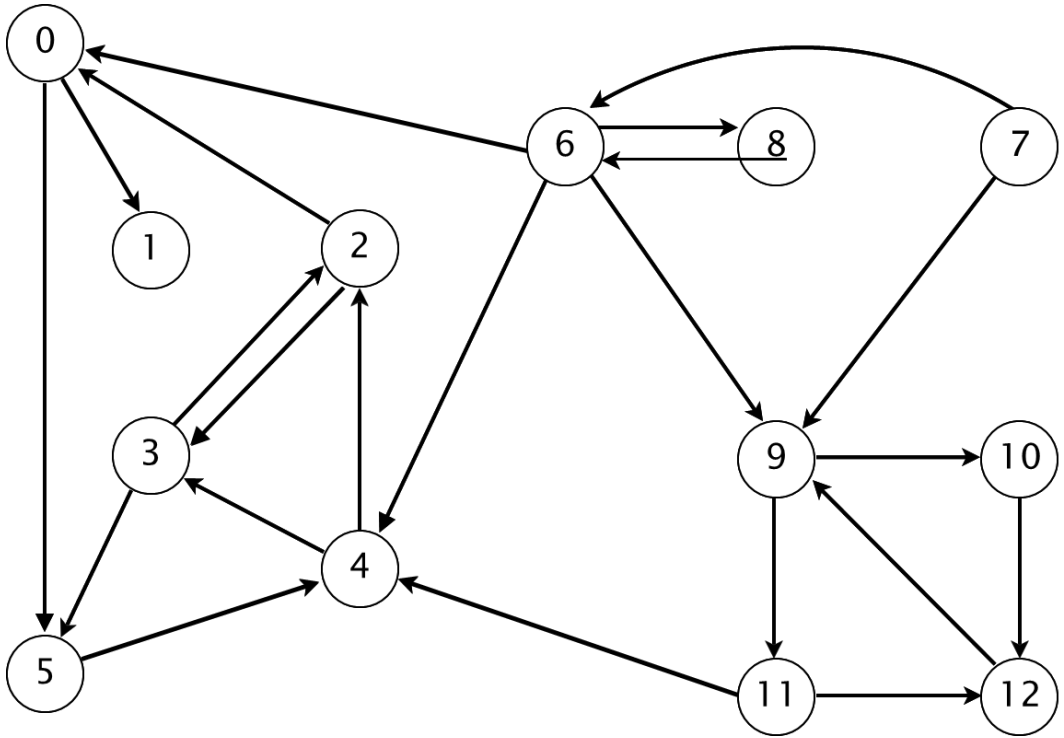
digraph G and its strong components



kernel DAG of G (topological order: A B C D E)

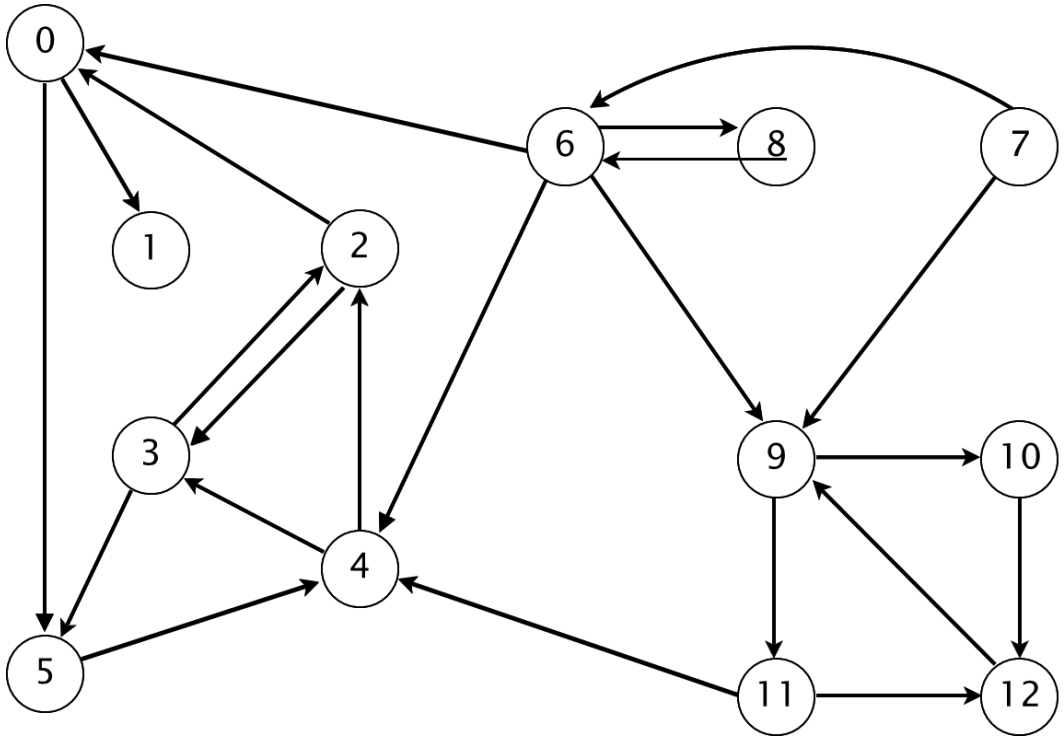
Kosaraju Algorithm Demo

- Compute reverse postorder in G^R
- Run DFS in G, visiting unmarked vertices in reverse postorder of G^R



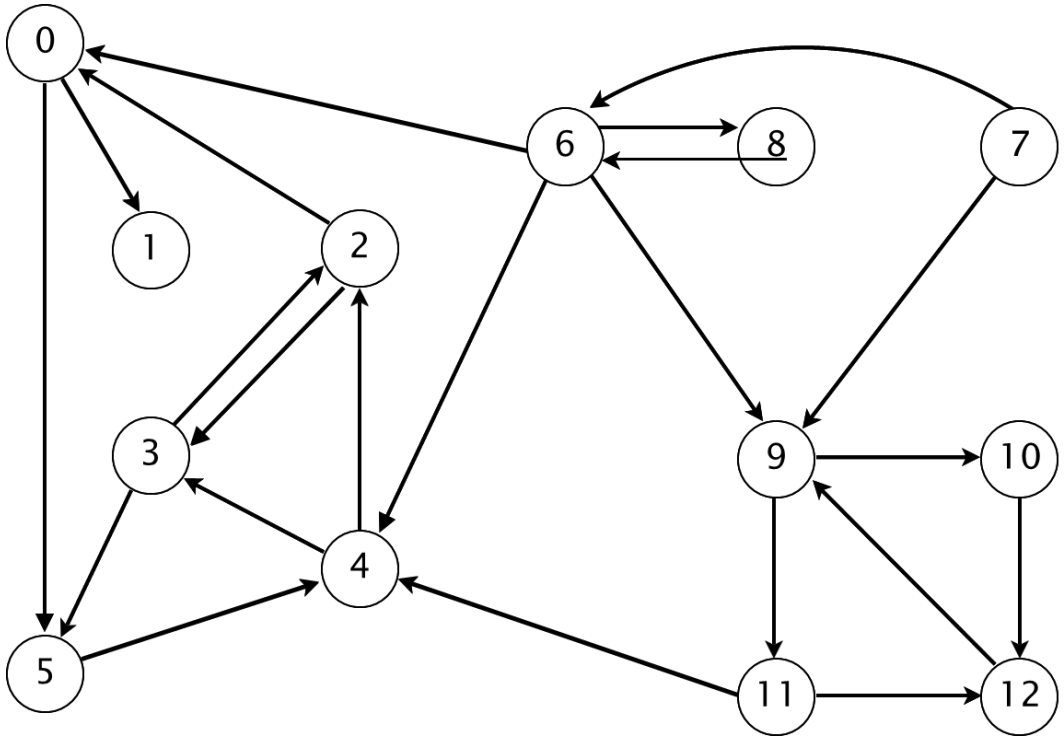
Kosaraju Algorithm Demo (1)

- Compute reverse postorder in G^R
  - **1 0 2 4 5 3 11 9 12 10 6 7 8**



Kosaraju Algorithm Demo (2)

- Run DFS in G, visiting unmarked vertices in reverse postorder of G^R
  - **1 0 2 4 5 3 11 9 12 10 6 7 8**



v	id[]
0	1
1	0
2	1
3	1
4	1
5	1
6	3
7	4
8	3
9	2
10	2
11	2
12	2

Kosaraju Algorithm Implementation

```
public class KosarajuSCC {
   private boolean[] marked;
   private int[] id;
   private int count;

   public KosarajuSCC(Graph G) {
        marked = new boolean[G.V()];
        id = new int[G.V()];
        TopologicalSort order = new TopologicalSort(G.reverse());
        for (int v : order.reversePostOrder()) {
            if (!marked[v]) {
                dfs(G, v);
                count++;
   } } }

   private void dfs(Graph G, int v) {
      marked[v] = true;
      id[v] = count;
      for (int w : G.adj(v)) {
         if (!marked[w]) {
            dfs(G, w);
   } } } ...
```

two lines difference compared to CC

In-Class Quiz 8

- What is the running time of the Kosaraju algorithm?
  - Please explain.

Thank you for your attention !

- **In this lecture, you have learned:**
  - to find connected components using DFS and answer connectivity queries in constant time
  - to do topological sort on a DAG
  - to find strongly connected components using Kosaraju algorithm
  - to implement and analyze them

- Please continue to work on **Exercise 11**